Universidad de Antioquia
Facultad de Ingeniería
Dpt. De Electrónica y de Telecomunicaciones

Análisis, diseño y solución Desafio II

Informatica II 2026-1

Realizado por:

Mateo Rodriguez Yepez
Emanuel Velez Arteaga

Docente:

Augusto Salazar

Abril 26, 2026

1 Análisis y Manejo de Datos

Al desarrollar una simulación y un análisis de la UdeAWorldCup, nos damos cuenta de que muchos de los datos que abarca el desafío tienen dependencias con otros datos. Debido a la restricción de librerías, es necesario contar con una solución que permita manejar datos que presenten estas dependencias y que, a la vez, se pueda acceder a ellos de manera fácil.

Nuestras principales fuentes de archivos serán aquellos con extensión .csv. A diferencia de los archivos de texto plano con extensión .txt, los archivos .csv utilizan el carácter "," como delimitador entre columnas. Esta estructura estandarizada facilita significativamente los procesos de extracción y manipulación de datos.

Acontinuación se planteara el informe con una linealidad de temas, los cuales encabezando serián las definiciones de las classes con sus metodos, y los headers con sus funciones.

2 Classes

2.1 Clase jugador

La clase jugador fue diseñada para representar la información individual de cada futbolista dentro del sistema de simulación Su objetivo principal es almacenar estadísticas personales y datos básicos necesarios para el desarrollo del torneo.
Cada objeto de esta clase contiene información relevante como:
Nombre, Apellido,dorsal , Partidos jugados ,Goles anotados ,Minutos jugados , Asistencias Tarjetas , amarillas ,Tarjetas rojas y Faltas.

Implementamos un constructor llamado jugador(), el cual se encarga de inicializar todos los atributos en 0, Teniendo en cuenta que la clase jugador es super importante debido a que con ella podemos simular mejor el comportamiento de las estadisticas de cada jugar en cada fase del torneo.

2.2 Clase equipos

La clase equipo fue diseñada para representar cada selección nacional participante en la simulación. Su función principal es almacenar la información general del país, estadísticas colectivas del torneo y la plantilla oficial de jugadores.
Esta clase cuenta con una estructura compuesta por variables informativas de tipo string y estadísticas numéricas de tipo int. Un aspecto sumamente importante dentro de esta clase es la plantilla de jugadores:
jugador* plantilla[26];
la cual consiste en un arreglo de tipo Jugador que nos permite manejar la memoria de forma dinámica.

Adicional tenemos el constructor equipo() y el destructor equipo() para limpiar la memoria.

2.3 Clase bombo

La clase Bombo se compone de un conjunto de equipos pertenecientes al mismo nivel de ranking dentro del sorteo. Internamente utiliza una Lista< *equipo∗* >, almacenando punteros hacia los objetos equipo ya cargados en memoria, evitando duplicación de datos. Expone el método mezclar( ) para aleatorizar el orden de sus equipos antes del sorteo, y permite el acceso por índice mediante operator[ ].
El torneo cuenta con cuatro bombos de doce equipos cada uno, conformados de la siguiente manera: el Bombo 1 contiene al anfitrión (Estados Unidos) más los once equipos de mayor ranking mundial, los bombos 2, 3 y 4 contienen los siguientes grupos de doce equipos en orden ascendente de ranking.

2.4 Clase grupo

La clase Grupo representa uno de los doce grupos del torneo. Su responsabilidad principal es controlar las restricciones de conformación: ningún grupo puede tener más de un equipo de la misma confederación, con excepción de la UEFA, que puede tener hasta dos representantes por grupo. Esta lógica se implementa en el método puedeAgregar(equipo*), que consulta internamente el método contarDeConf( ) antes de permitir la inserción de un equipo.

2.5 Clase Sorteo

La clase Sorteo orquesta todo el proceso. Gestiona un arreglo de cuatro punteros a Bombo y doce punteros a Grupo, y expone los siguientes métodos principales:

- conformarbombos(equipo* torneo[ ], int total): recibe el arreglo de equipos cargados desde los archivos CSV, separa al anfitrión, ordena el resto por ranking mediante inserción directa y distribuye los equipos en los cuatro bombos.
- realizarSorteo( ): ejecuta el método privado intentarSorteo( ) de forma iterativa hasta obtener una distribución válida o agotar el límite de 10.000 intentos.
- imprimirbombos( ) e imprimirGrupos( ): muestran por consola la conformación final.

2.6 Clase Memoria

Para asegurar que el proyecto cumpla con los requisitos propuestos en la guia del documento. Se implemnta un sistema de monitoreo llamado clase de **memoria**. Esta clase funciona para la regulación de cuantas iteraciones realiza el programa y cuanta memoria RAM esta consumiendo el programa en cada estapa clave.
Ya sea para la carga de datos CSV, al hacer estas tareas, el sistema imprime en la consola un pequeño resumen donde muestra: Bytes consumidos e

iteraciones realizadas. De esta manera, garantizamos y demostramos, que el simulador no solo hace su trabajo, sino que tambien administra correctamente los recursos del sistema.

2.7 Clase Partido

La clase Partido fue diseñada para representar y gestionar la simulación completa de un encuentro de fútbol dentro del torneo. Esta clase actúa como una entidad central del sistema, ya que conecta a dos objetos de tipo equipo (local y visitante) y administra tanto la información logística del partido como el desarrollo estadístico del mismo. Entre los datos almacenados se encuentran la fecha, la hora, la sede, el cuerpo arbitral, el marcador final y los porcentajes de posesión de balón.
un componente relevante es simularJugadores(), método que recorre la alineación titular de cada equipo y actualiza estadísticas individuales como partidos jugados, minutos disputados, goles, faltas y tarjetas. Esto permite que la simulación no solo produzca un marcador global, sino también un seguimiento detallado del rendimiento de los jugadores durante el torneo.

La clase dispone de dos modos principales de ejecución: simular(), utilizado para partidos de fase de grupos donde se contemplan victorias, derrotas o empates; y simular eliminatoria(), orientado a rondas directas donde necesariamente debe existir un ganador. En caso de empate, este último contempla tiempo suplementario y un mecanismo de desempate basado en probabilidades asociadas al ranking FIFA.
En términos de Programación Orientada a Objetos, la clase Partido evidencia encapsulamiento al concentrar en una sola estructura los datos y operaciones relacionados con un encuentro. Además, demuestra reutilización de código al emplear métodos internos para ambas modalidades de juego. Gracias a esta implementación, el sistema logra simular partidos de forma automática, coherente y estadísticamente fundamentada.

3 Headers y Funciones

3.1 Clases.h

Este archivo de cabecera es uno de los primeros que se crean en el proyecto. En él se encuentra la inicialización de las clases Jugador y Equipo (cuya información detallada se presentó en la sección 2 y diagrama UML).
Asimismo, este header contiene una función denominada repartir goles, la cual se encarga de asignar a cada jugador los goles correspondientes a su equipo, distribuyéndolos de manera uniforme mediante los operadores de división entera y división módular. De esta forma, se reparte una cantidad base igual para todos y el residuo se asigna de a uno hasta agotarlo.

3.2 Eliminatorias.h

El archivo de cabecera Eliminatorias.h es el módulo donde concentramos los procedimientos que nos aproximan a una simulación precisa de las fases eliminatorias. En este header se implementan las siguientes funciones clave:

bool es mejor(equipo* a, equipo* b)

Esta función actúa como un filtro de comparación entre dos equipos. Determina cuál de los dos es superior con base en los criterios de mayor cantidad de puntos y, en caso de empate, diferencia de goles.

void clasificar r16(Sorteo sorteo, Equipo* clasificadosR16[32])

El propósito de esta función es determinar qué equipos avanzan a los octavos de final, siguiendo los pasos que se detallan a continuación:
Recorremos los 12 grupos de la fase previa y copiamos los equipos en un arreglo temporal de tipo Equipo denominado tabla.
Organizamos los equipos dentro de cada grupo de modo que queden ordenados del mejor al peor, empleando para ello el método de bubble sort.
Aplicamos las condiciones de clasificación: el equipo en la posición [0] y el de la posición [1] de cada grupo clasifican de manera directa a la siguiente ronda.
Posteriormente, para determinar los mejores terceros:
Creamos una lista temporal con todos los equipos que ocuparon el tercer puesto en sus respectivos grupos.
Ordenamos nuevamente esta lista mediante bubble sort.
Seleccionamos los 8 mejores terceros, es decir, aquellos ubicados en las posiciones [0] a [7] tras la ordenación.

void simularRonda(Equipo* equiposIn[], int numEquipos, Equipo* equiposOut[], ...)

En esta función se materializa el polimorfismo del torneo, ya que permite simular cualquier fase eliminatoria (R16, R8, R4, R2) sin necesidad de reescribir la lógica para cada una.
La simulación sigue un proceso sencillo:
Enfrentamos a los equipos de dos en dos.
Seleccionamos de manera aleatoria los árbitros y las sedes correspondientes al encuentro.
Instanciamos un objeto de la clase Partido, que almacena toda la información del encuentro.
Simulamos el partido hasta obtener un ganador.
Almacenamos al ganador en el arreglo de salida (equipos out), que representa la nómina de clasificados a la siguiente ronda.

Por ultimo unicamente nos queda las funciones jugar fases finales y mostrar resumen mundial en donde es una funcion que implementamos para que se viera llamativa mediante terminal, esta funcion simula con ayuda de la función simular ronda cada fase correspondiente del torneo.

3.3 Funciones auxiliares.h

Este bloque de código es el encargado de la lectura, carga, almacenamiento y actualización de los archivos .csv.
Nuestra primera función en este módulo se ocupa de extraer cada palabra del archivo .csv. Esta función es sumamente importante porque una de nuestras ideas centrales es mantener una relación estrecha entre la estructura de nuestras clases y el formato de los archivos csv.

Las funciones siguientes, tales como cargar datos y cargar confederaciones, siguen una lógica muy similar. Nuestro objetivo es transformar los datos contenidos en los archivos csv en clases o estructuras de datos que nos permitan trabajar con ellos de forma dinámica.
Finalmente, lo que hacemos es volcar esas estructuras dinámicas de datos de vuelta a los archivos .csv. Cabe señalar que, si en algún momento necesitamos

incorporar nuevos datos desde el csv, simplemente creamos funciones similares que nos permitan cargar esos datos adicionales. En última instancia, guardar los datos equivale a sobreescribir los propios archivos con la información actualizada.

Visto Gráfica de los couts del header

3.4 Math.h

La idea central de este header, es la implementación de la formula dada para la simulación de goles del torneo la cual es:

$$\lambda_A = \mu \cdot \left(\frac{GF_A}{\mu}\right)^{\alpha} \cdot \left(\frac{GC_B}{\mu}\right)^{\beta}$$

Formula Pdf Guía

Lo primero que debemos determinar si un evento ocurre o no ocurre, nosotros claramente dandole una probabilidad conceptual data, es decir que si quiero que para que un evento pase, debe superar cierto rango y eso la hace la funcion bool probabilidad, permitiendome tener mas control acerca si quiero que halla gol o no.

Las siguiente funciones matematicas, son una forma mas cásera de hallar cierto valores matematicos sin necesidad de librerias externar, en este caso buscamos un valor medio tal que medio elevado a la 5 = x, es decir utilizamos las definiciones matematicas básicas para hallar algo, en este caso una busqueda binaria para hallar eso mediante 50 iteraciones, Cálculando asi las raicez quinta

La siguiente funcion es acerca de calcular x elevedo a (alpha) , en este caso alpha = 0.6 asi que utilizamos la definicíon matematica

$$x^{0.6} = \sqrt[5]{x^3}$$

Lo mismo aplicamos para la función siguiente:

$$x^{0.4} = \sqrt[5]{x2}$$

3.5 Simulacion.h

En este archivo de cabecera definimos una clase que surgió de la necesidad y el moldeamiento progresivo de la simulación: la clase Partido. Esta clase constituye una de las unidades centrales de la simulación, ya que sobre ella recae la responsabilidad de gestionar las estadísticas de cada encuentro. En otras palabras, es el núcleo de la simulación.
Dada su importancia, la clase Partido es rica en métodos, entre los cuales se destacan:
Calcular la posesión, tanto para el equipo local como para el visitante.
Calcular los goles esperados, para lo cual se invocan las funciones implementadas en el header Math.h.
Simular los goles a favor y los goles recibidos por cada equipo, apoyándose igualmente en las funciones matemáticas definidas previamente.

Contamos también con un método que nos permite simular el desempeño individual de los jugadores titulares. Durante esta simulación, las estadísticas de cada jugador deben incrementarse de acuerdo con lo ocurrido en el partido. Aquí es donde confluyen los demás headers y clases del proyecto, y todas estas simulaciones se llevan a cabo mediante probabilidades aleatorias.

El método simular nos permite ejecutar la simulación completa de la fase de grupos, generando todas las estadísticas e informaciones necesarias del partido. Adicionalmente, disponemos de otra función denominada simular eliminatorio, que representa un polimorfismo conceptual entre ambas simulaciones, encapsulando los métodos correspondientes y permitiendo reutilizar la lógica tanto en fase de grupos como en rondas eliminatorias.

4 Manejo de Datos para Asignación Uniforme de Goles

La primera tarea fundamental que abordamos fue definir el método de procesamiento de los datos contenidos en los archivos .csv. Inicialmente, el

archivo jugadores.csv posee estadísticas fijas y predefinidas. Nuestro objetivo es modificar dichas estadísticas tomando como referencia los datos de goles almacenados en un segundo archivo "equipos.csv". El procedimiento consiste en acceder a la cifra total de goles por equipo registrada en "equipos.csv" y, con base en esa información, redistribuir y asignar de manera uniforme esos goles entre todos los jugadores que conforman las plantillas correspondientes dentro del archivo "jugadores.csv".

4.1 Classes Para el Manejo de Datos

Para garantizar una correcta manipulación de los datos y, en particular, el manejo de los archivos de almacenamiento con extensión .csv, es imprescindible mantener una compatibilidad estricta entre la estructura de los archivos y los atributos definidos en nuestras clases del modelo de datos. En este sentido, debemos tener presente que el formato establecido para el archivo "jugadores.csv" es el siguiente:

equipo	nombre	apellido	dorsal	PJ	goles	minutos jugados	asistencias	t.amarillas	t.rojas	faltas
France	Mike	Maignan	1	0	0	0	0	0	0	0
Spain	Alex	Remiro	12	0	0	0	0	0	0	0

Y el formato definido para el archivo "equipos.csv" es el siguiente:

pais	dt	ranking	ga	gc	pg	pe	pp	t.amarillas	t.rojas	faltas
France	DidierDeschamps	1	20	5	7	1	0	0	0	0
Spain	Luis de la Fuente	2	20	5	7	1	0	0	0	0

Una vez definido el formato de los archivos, nuestras clases adoptarán una estructura estrechamente alineada con dicha disposición. Para el manejo de los datos, emplearemos variables de tipo int para los valores numéricos y de tipo string para aquellos campos que requieran cadenas de caracteres. En consecuencia, dado que contamos con dos archivos "jugadores.csv" y "equipos.csv", implementaremos dos clases correspondientes: equipo y jugador. Cada una de estas clases contendrá los atributos necesarios para representar fielmente las columnas de los archivos y facilitar así su posterior lectura y escritura.

4.2 Asignación Uniforme de Goles

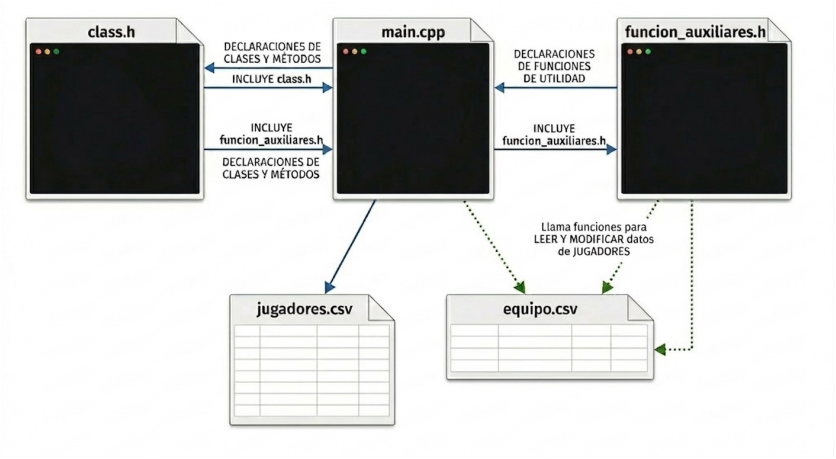
Para llevar a cabo la asignación de los goles, el primer paso consiste en extraer cada uno de los datos contenidos en los archivos .csv. Con este propósito, implementamos una función denominada "extraer palabra", ubicada en el header "funciones auxiliares.h". Dicha función se encarga de recorrer el formato del archivo y aislar cada valor delimitado por comas, devolviéndolo como una variable de tipo string. De esta manera, realizamos una extracción secuencial de los campos que posteriormente serán convertidos al tipo de dato

correspondiente (int o string) según lo requiera cada atributo de nuestras clases.

equipo → nombre → apellido → dorsal ...

A continuación, dentro del mismo header, implementamos otra función denominada "cargar datos". Esta función se encarga de leer y procesar el contenido de los archivos, transformando los valores extraídos previamente como string hacia los tipos de dato correspondientes: int, string o punteros a objetos de las clases equipo y jugador. Una vez finalizada la carga, se invoca desde el archivo class.h la función "repartir goles". Esta última recorre la lista de jugadores y distribuye equitativamente los goles correspondientes a cada equipo mediante una combinación de división entera y división módular, garantizando así un reparto lo más uniforme posible entre los integrantes de la plantilla.

Por ultimo tenemos la función "guardar datos" cuya función es reescribir datos en el csv original.



Ejemplo gráfico del acceso para la modificación y extracción de datos de los csv

5 Listas enlazadas

Para la realización de la siguiente idea dada en la sección del sistema de bombos, grupos y sorteo, se diseño la siguiente ruta de diseño con enfoque listas enlazadas y nodos.

5.1 Nodos y listas enlazadas genericas

Se implementó una lista enlazada simple mediante las clases Nodo$< T >$ y Lista$< T >$, utilizando su uso para cualquier tipo de dato. La clase Lista$< T >$ expone operaciones fundamentales de la siguiente manera: agregar, eliminar, acceso por índice mediante operator[ ], la clase mezclar se basa en el algoritmo Fisher-Yates para aleatorización y vaciar. Se eligió esta estructura sobre arreglos estáticos para permitir la manipulación dinámica de los equipos durante el proceso de sorteo sin desperdiciar memoria.

6 Procesamiento y desarrollo

Durante el proceso de desarrollo del desafío y en el análisis conjunto que realizamos con mi compañero, intentamos seguir el flujo lineal propuesto en la guía en PDF. Sin embargo, pronto nos dimos cuenta de que necesitábamos aplicar una estructuración más organizada para entrelazar todos los componentes y mantener un control óptimo de las funciones, los archivos de cabecera y la organización de los datos. Ahora que ya tenemos claras las bases expuestas en las secciones 1 y 2, podemos comenzar a describir las demás partes del desarrollo

6.1 Linealidad del Desarrollo

Una vez cargados los datos de entidades como equipos, árbitros, sedes y confederaciones las cuales comparten la misma estructura de lectura y escritura en los archivos .csv, nuestra siguiente tarea consiste en realizar los sorteos. Es por ello que a continuación pasamos a detallar:

6.2 Lógica del sorteo

El método intentarSorteo( ) implementa el proceso de sorteo en dos etapas:
Etapa 1 - Bombo 1: Se crea una copia del Bombo 1 y se mezcla aleatoriamente. El equipo anfitrión (Estados Unidos) se asigna directamente al Grupo A. Los once equipos restantes del Bombo 1 se distribuyen uno a uno en los grupos B hasta L, garantizando que cada grupo tenga exactamente un equipo cabeza de serie.

Etapa 2 - Bombos 2, 3 y 4: Para cada bombo, se crea una copia mezclada aleatoriamente. Por cada equipo se construye una lista de grupos válidos, es decir, aquellos que ya tienen exactamente b equipos asignados (donde b es el índice del bombo actual) y que cumplen la restricción de confederación. Se elige aleatoriamente uno de los grupos válidos y se asigna el equipo. Si para algún equipo no existe ningún grupo válido, el intento falla y se reinicia el proceso completo. Este enfoque de reintentos garantiza que el resultado final siempre respete todas las restricciones del torneo, al costo de posibles múltiples iteraciones en casos de alta concentración de equipos de la misma confederación.
Para gestionar los equipos durante el sorteo, decidí usar las listas enlazadas (mediante nuestras clase *lista* <>) en lugar de arreglos de tamaño fijo. La razón de esto en para que durante el proceso del sorteo, estamos sacando equipos constantemente de los bombos para asignarlos a los grupos. Una lista enlazada permite dque agregar y eliminar estos datos sea un proceso continuo y eficiente.
Además, cuando necesitamos ordenar las tablas de posiciones de los grupos, optamos por usar el algorimo que organiza de forma burbuja (Bubble sort) aplicado para esté ordenamiento de 4 equipos por grupo se da más fácil y ayuda a no sobrecargar con algoritmo pesados.

6.3 Clase tabla de Grupo - tablas[i]

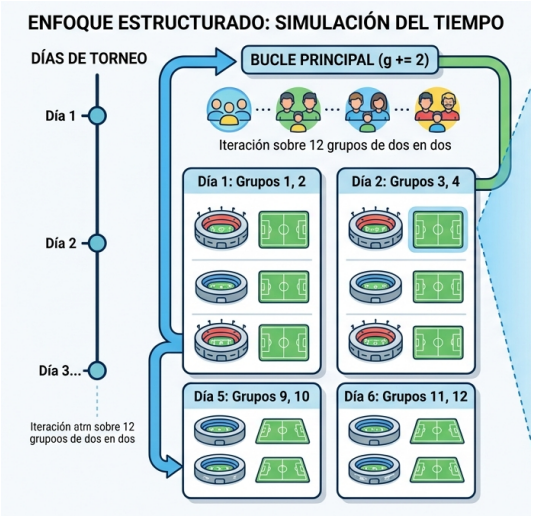
Para manejar las posiciones, creamos la clase **tabladegrupo**. Como cada grupo del mundial está compuesto, por 4 equipos, esta clase nos ayuda a reservar un espacio fijo dpara ellos. Además del metodo inicializador,

inicializar(Grupo* g , char) tiene como función: Tomar esos 4 equipos y los inscribe en la tabla justo antes de que se empiece a simular los equipos. Por otro lado, utilizamos la clase de **filadeclasificacion**. Actúa como registro individual de cada equipo dentro de la tabla del grupo, llevandola cuenta de actualizada de cuántos puntos han ganado, los goles que han marcado a favor y los goles que han recibido.

6.4 Fase de Grupos

La guía en el PDF establece una limitación de 17 días para la duración del torneo. Sin embargo, al analizar matemáticamente este plazo en conjunto con los demás requisitos estipulados tales como la cantidad de partidos por día y la frecuencia de juego de cada equipo, llegamos a la conclusión de que resultaba insuficiente. Esta situación nos obligó a extender el tiempo estipulado para la fase de grupos a 18 días.

La fase de grupos constará de 3 jornadas, las cuales actuarán como el iterador principal de esta etapa del torneo. A continuación, debemos simular el cruce de grupos, asegurándonos de que en cada fecha se enfrenten dos grupos diferentes. Una vez definida la iteración entre los grupos, el siguiente paso es iterar entre los equipos dentro de cada grupo, prestando especial cuidado a no perder el rastro de los índices. Esto nos permite evitar posibles duplicaciones de partidos y respetar la restricción de que el enfrentamiento se limite a los equipos ubicados entre la posición 1 y la 4 de cada grupo.



Patrón gráfico de jornadas

Luego de las iteraciones iniciales, corresponde simular un todos contra todos respetando 1 condición y es que todos juegan 3 partidos sin repetir rivales utilizando el siguiente patrón:

Jornada 0: Equipo 1 vs Equipo 2 | Equipo 3 vs Equipo 4.

Jornada 1: Equipo 1 vs Equipo 3 | Equipo 2 vs Equipo 4.

Jornada 2: Equipo 1 vs Equipo 4 | Equipo 2 vs Equipo 3.

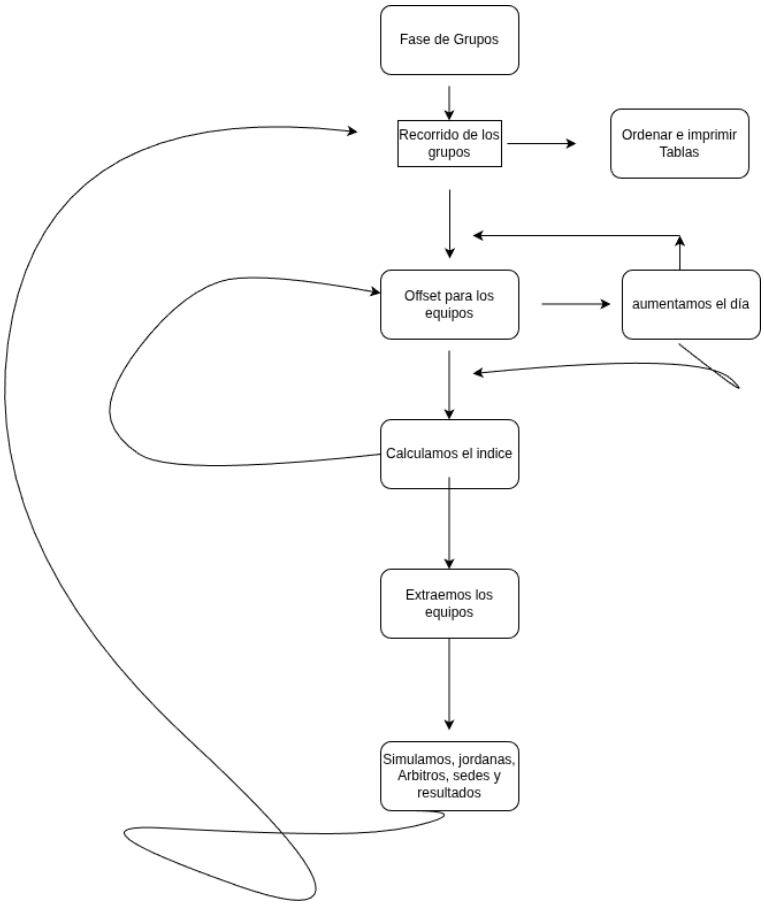
Ejemplo gráfico de simulación de grupos del torneo

Posteriormente, y con el objetivo de aportar mayor realismo a la simulación además de cumplir con un requisito establecido, asignamos de manera aleatoria

un equipo arbitral compuesto por 3 árbitros(cada uno con un código diferente), así como una de las 9 sedes disponibles donde se disputará el encuentro

A continuación, procedemos a instanciar una de nuestras clases principales: la clase Partido, pasándole todos los parámetros requeridos (la definición completa de todas las clases se encuentra en la sección 2 de este documento). Luego, invocamos nuestro método invocar (los métodos también fueron descritos en secciones anteriores). Internamente, esta llamada desencadena la ejecución del método registrar resultados, el cual se encarga de almacenar de forma persistente las estadísticas del partido, una funcionalidad que constituye una parte fundamental del desafío.

Por último, al finalizar la simulación de la fase de grupos, invocamos el método ordenar, el cual se encarga de ubicar a los equipos dentro de cada grupo conforme a las especificaciones de posiciones y a los criterios de desempate establecidos.



Aproximación con Diagrama a la fase de grupo mediante esquema hecho en draw.io

6.5 Clasificación R16

Una vez que los 32 equipos clasificados están organizados en un arreglo de tipo Equipo, utilizamos una función denominada clasificarR16. Esta función evalúa el objeto Sorteo y selecciona los equipos que, de acuerdo con un criterio numérico como los puntos obtenidos o la diferencia de goles, obtuvieron la clasificación a la siguiente ronda.

6.6 Clasificaciones R8 R4 R2 y R1

Seguido de esto inicializamos una variables que serán de vital importancia en nuestra simulación , que son los puestos 1,2,3 y 4, asi como tambiente arreglos tipo equipo que serviran como parametro para una función muy importante, en este caso la funcion jugar fases finales.

Esta función es especialmente importante porque puede interpretarse como una función que exhibe polimorfismo. Es decir, a partir de ella pueden derivarse o realizarse otras operaciones muy similares. Esto se debe a que las fases posteriores R8 (octavos de final), R4 (cuartos de final), R2 (semifinal) y R1 (final) comparten una estructura de simulación prácticamente idéntica, lo que permite reutilizar y adaptar la misma lógica con modificaciones mínimas.

En nuestro caso, no implementamos funciones separadas para R2 (semifinal) y R1 (final), ya que la única diferencia frente a las rondas anteriores es la disminución en la cantidad de equipos, mientras que la lógica de simulación de cada enfrentamiento sigue siendo exactamente la misma.

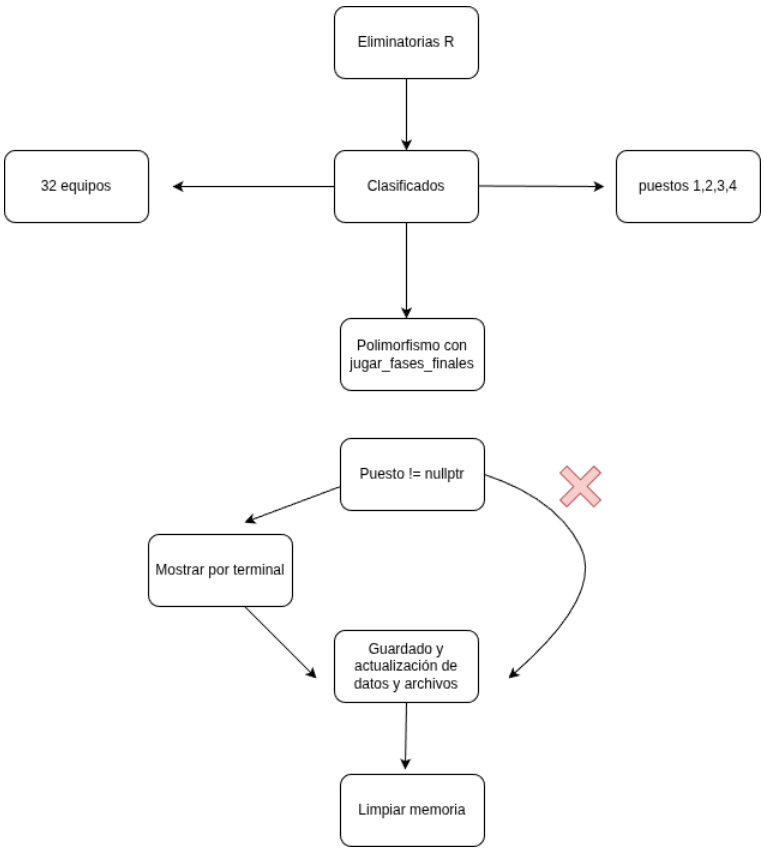
6.7 Simulación Tercer Puesto

Para simular el partido por el tercer puesto, lo único que hacemos es instanciar un nuevo objeto de la clase Partido con los parámetros correspondientes para esa fase y, a continuación, ejecutar la simulación. De esta manera, aprovechamos el polimorfismo que buscamos implementar a lo largo de este desarrollo, ya que la lógica de simulación se reutiliza sin necesidad de modificaciones adicionales.

6.8 Datos e Informes de Simulación

Finalmente, llevamos a cabo una verificación para comprobar que la simulación del mundial se haya ejecutado correctamente. Esto lo logramos de manera sencilla: verificando que el equipo ubicado en la posición 1 del arreglo no sea un puntero nulo (nullptr), evitando así que potencialmente se convierta en un puntero salvaje que comprometa la integridad del programa.

Otro aspecto muy importante a tener en cuenta es que, en las fases eliminatorias (R8, R4, R2 y R1), si la simulación arroja un empate, procedemos a desempatar mediante una lógica de programación sencilla: avanza a la siguiente ronda el equipo que tenga mayor peso, es decir, aquel que posea el mejor ranking FIFA y le asignamos un gol



Aproximación con Diagrama a Las Fases R